Forecasting Household Appliance Energy Consumption:

A Comparison of Statistical, Machine Learning, and Foundation-Model Approaches

GitHub

1. Introduction

Accurate short-term forecasting of household appliance energy consumption supports demand-side management, load balancing, and personalized energy-saving feedback for occupants. This report examines 24-hour-ahead forecasting of appliance energy use from the UCI Appliances Energy Prediction dataset, benchmarking four distinct modeling approaches under a shared, fair evaluation setup: simple statistical benchmarks, a classical seasonal ARIMA model (SARIMA), a feature-based gradient-boosted tree model (XGBoost), and a pretrained time-series foundation model (Chronos-Bolt-Small) used zero-shot.

Three main questions guide the study: (1) does explicitly modelling seasonality offer a meaningful advantage over naive extrapolation methods; (2) can a flexible machine-learning approach with engineered features outperform a properly specified statistical model; and (3) how does a general-purpose foundation model, with no prior exposure to this data, perform relative to both. Every model is tested on the same held-out 14-day test period using the identical metrics, enabling a consistent comparison of predictive performance.

2. Data and Exploratory Analysis

The dataset contains 19,735 observations at 10-minute resolution over ~4.5 months (Jan–May 2016) from a low-energy house in Belgium, comprising the target variable (Appliances, Wh), a lights sub-metering channel, nine indoor room temperature/humidity sensors, six outdoor weather variables, and two random noise columns (rv1, rv2) included as controls. The data contains no missing values and no time gaps. We resampled to hourly resolution (summing energy columns, averaging sensor/weather columns), giving 3,290 hourly observations.

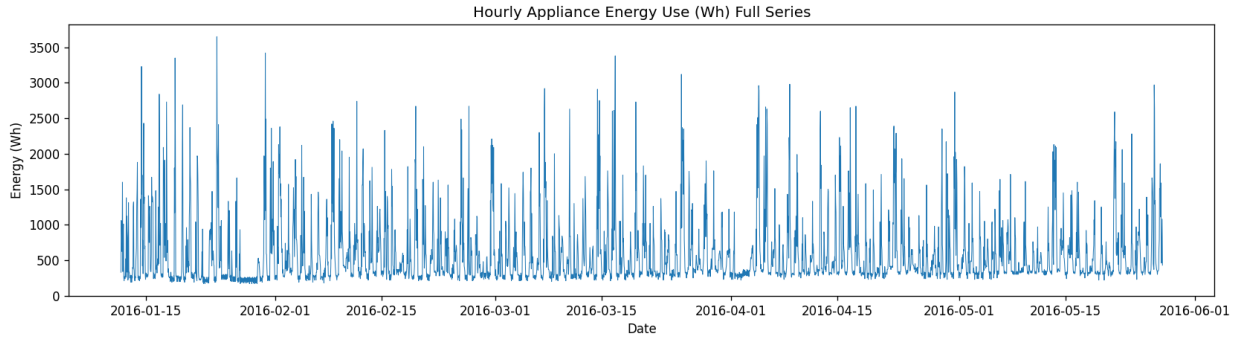

Figure 1: Hourly appliance energy use across the full observation period

The series exhibits a strong repeating daily cycle with no long-run trend: consumption dips overnight (02:00–05:00) and peaks in the early evening (17:00–19:00), consistent with typical occupant activity patterns (Figure 2, left). A secondary, weaker weekly pattern is also present, with Mondays showing the highest average use and mid-week the lowest (Figure 2, right).

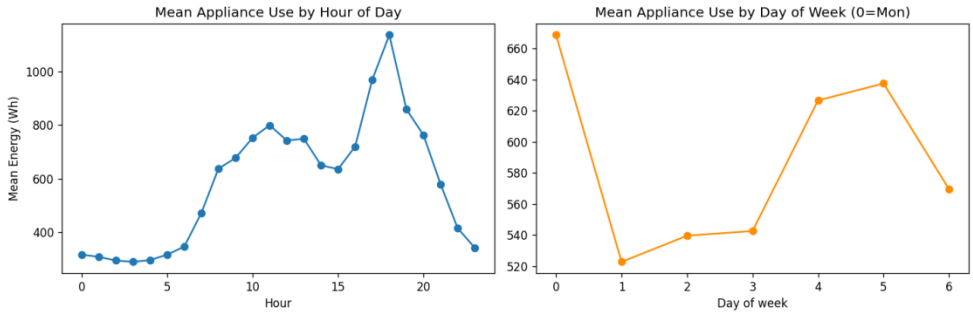

Figure 2: Mean appliance energy use by hour of day (lek) and day of week (right, 0=Monday.

An additive seasonal decomposition (period = 24h) confirms a dominant, stable daily seasonal component with no discernible long-term trend, and a residual component that captures the large, somewhat irregular usage spikes. Stationarity was assessed with the Augmented Dickey-Fuller (ADF) and KPSS tests. Both tests agree the raw hourly series is already stationary in levels (ADF $p \approx 0.00$, KPSS $p \approx 0.10$), i.e. mean-reverting with no unit root. However, the ACF (Figure 3) shows a slow, oscillating decay with repeated spikes at lags 24, 48 and 72 the signature of seasonal, rather than trend, structure. This motivated using a seasonal difference (D=1, s=24) in the SARIMAX specification despite the series already passing standard unit-root tests, since seasonality and non-stationarity in the classical unit-root sense are distinct properties.

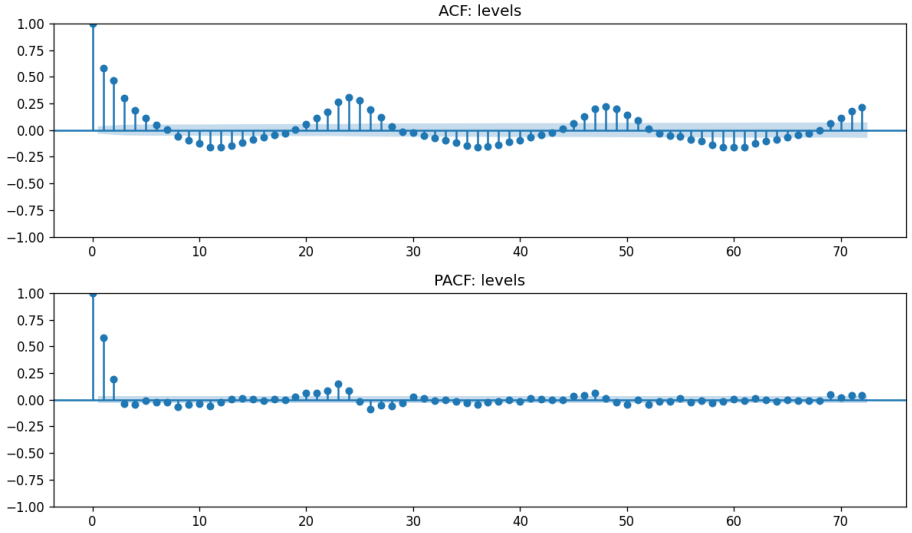

Figure 3: ACF and PACF of the hourly Appliances series, showing seasonal spikes at multiples of 24.

3. Forecasting Problem Definition

The target variable is Appliances (hourly Wh). The forecast horizon is fixed at 24 hours ahead, matching the assignment specification. We use a single chronological train/test split: the final 14 days (336 hours) are held out as the test period (common to every model), with all preceding data available for training/fitting. To avoid relying on a single, possibly unrepresentative 24-hour window, every model is evaluated with a rolling protocol: the 24-hour forecast is repeated at 14 non-overlapping daily origins spanning the full test period, and metrics are averaged across all 14 windows. This rolling average is used as the main comparison score throughout this report; single-window results (matching the assignment's literal instruction) are reported alongside where relevant. Four complementary metrics are reported: RMSE and MAE (in Wh, capturing absolute error magnitude, with RMSE weighting large errors/spikes more heavily), and MAPE and sMAPE (scale-free percentage-based errors that are easier to interpret but less stable when actual values approach zero).

4. Methods

4.1 Benchmark models

Five simple benchmarks were implemented: Mean (overall training mean), Naive (last observed value), Daily Seasonal Naive (value 24h earlier, repeated), Weekly Seasonal Naive (value 168h earlier, repeated), and Drift (naive plus a linear extrapolation of the average historical trend).

4.2 SARIMA

A Seasonal ARIMA model was fitted to the Appliances series alone (no exogenous regressors see Section 4.3 and Discussion Q2 for justification). The seasonal order was fixed at (P,D,Q,s) = (1,1,1,24) based on the seasonal structure identified in Section 2. The non-seasonal order (p,d,q) was selected by an exhaustive AIC grid search over p∈[0,6], d∈[0,2], q∈[0,6] 147 combinations fitted on a recent 30-day subset of the training data for computational tractability, with the winning order refit on the full ~3,000-hour training set. The best model found was SARIMA(1,0,6)×(1,1,1,24).

4.3 Feature engineering and covariates

For the machine-learning model, the existing sensor/weather covariates were supplemented with cyclical time encodings (sine/cosine of hour-of-day and day-of-week, plus a weekend flag) and lag/rolling features of the target (lags at 1, 2, 3, 24, 48, and 168 hours; rolling mean/std at 24h and rolling mean at 168h, computed strictly from past values to avoid target leakage). The XGBoost implementation also retained the original hour, dayofweek, rv1, and rv2 columns in its actual training feature matrix. A separate cleaned feature matrix was generated for reference but was not used for the reported XGBoost results. A correlation check (Figure 4) found every individual sensor/weather variable only weakly correlated with Appliances ($|r| \leq 0.193$; approximately ≤ 0.19), consistent with the original dataset's own findings that occupant behaviour not ambient conditions primarily drives appliance use. These variables were nonetheless retained for the tree-based model, which can exploit non-linear interactions beyond simple linear correlation.

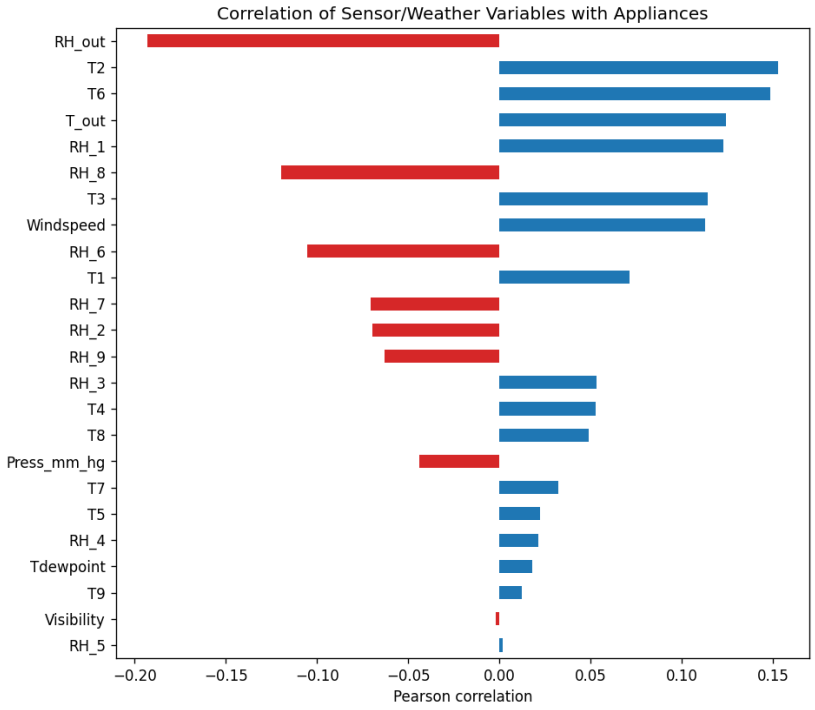

Figure 4: Correlation of sensor/weather covariates with Appliances; all are weak

4.4 Feature-based ML model (XGBoost)

Multi-step forecasting with a tabular model requires an explicit strategy. Two were tried and compared: (a) a recursive strategy, training one one-step-ahead model and feeding each prediction back as the next step's short-lag features; and (b) a direct multi-horizon strategy, training 24 separate models, one per horizon step, each predicting directly from the feature row available at the forecast origin. The recursive implementation uses future feature rows and therefore assumes future sensor/weather covariates are available, whereas the final direct strategy uses only the origin-time feature vector and does not access future test-set covariates. The direct strategy was clearly superior and was adopted as the final model. Both strategies used XGBoost regressors with 300 trees, max depth 4–5, and learning rate 0.05.

4.5 Foundation model (Chronos)

Chronos-Bolt-Small, a pretrained time-series foundation model, was used in a strict zero-shot setting: no fine-tuning or training on this dataset was performed. At each rolling-window origin, the model was given all real history available up to that point (an expanding context) and asked to forecast the next 24 hours directly via its native quantile-forecasting interface, with the median quantile taken as the point forecast.

5. Results

5.1 Benchmark results

Table 1 shows the rolling 14-window average for all five benchmarks. No single benchmark dominates every metric: Mean achieves the lowest RMSE (391.2 Wh) by always predicting a middling value, which minimizes squared error against a spiky series but is the least behaviourally useful forecast; Weekly Seasonal Naive achieves the best MAE, MAPE and sMAPE (254.6 , 37.1% , 32.5%), suggesting weekly timing of occupant behaviour is a stronger typical-case predictor than 24h-lag alone. Naive and Drift are clearly worst (RMSE ≈ 585), since anchoring on the last observed value poorly predicts a series that swings by 5-10× within a day.

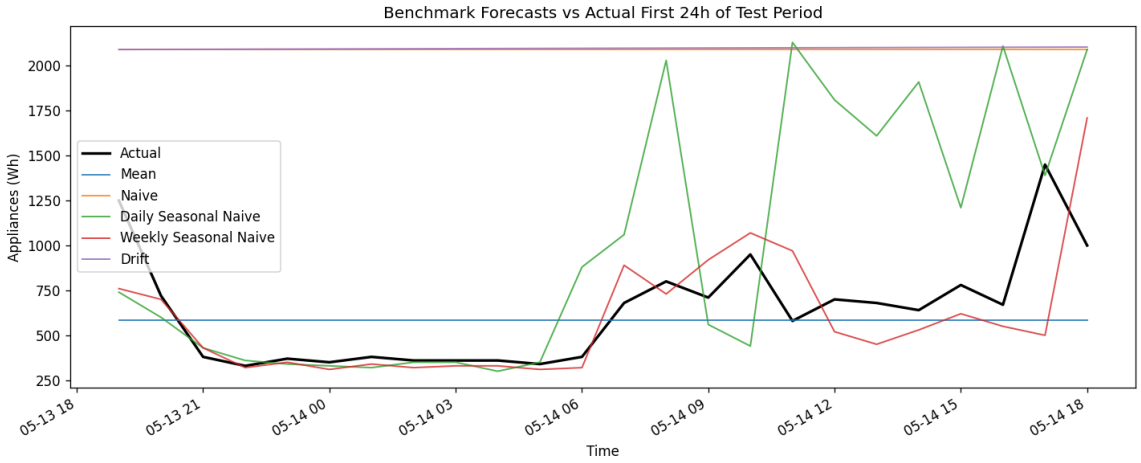

Figure 5: Benchmark forecasts vs. actual for the first 24h test window

5.2 SARIMA results

The final SARIMA(1,0,6)×(1,1,1,24) model converged cleanly. Residual diagnostics confirm a good fit: the residual ACF shows no significant autocorrelation at any lag (including 24/48), and the Ljung-Box test fails to reject white noise at both lags (p ≈ 0.38, 0.32). Residuals are, however, non-normal (heavy-tailed, right-skewed), meaning point

forecasts are trustworthy but the Gaussian 95% confidence interval understates spike risk and can extend below zero physically impossible for energy consumption (Figure 6).

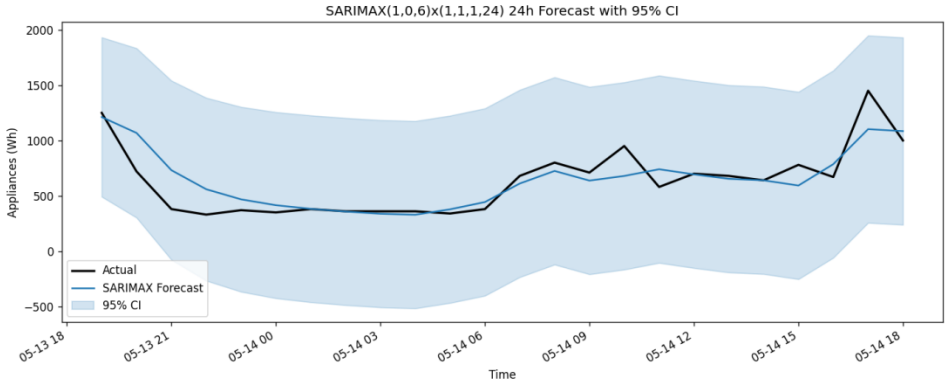

Figure 6: SARIMAX 24h forecast with 95% confidence interval, first test window

5.3 XGBoost results

The recursive strategy underperformed even the benchmarks (RMSE 394.5 Wh) due to compounding error over the 24-step horizon. The direct multi-horizon strategy corrected this, achieving RMSE 341.2 Wh beating every simple benchmark on RMSE and MAE. Unlike the recursive implementation, the final direct strategy generates each 24-hour forecast from the single feature vector available at the forecast origin, so it does not rely on observed future weather or indoor-sensor values. Feature importance (Figure 8) shows lag_1 and cyclical hour-of-day encodings dominate, with weekly features and seasonal lags also contributing meaningfully; individual weather/sensor variables rank far lower, consistent with Section 4.3's correlation analysis.

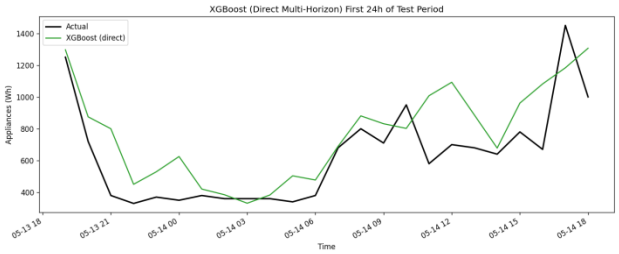

Figure 7: XGBoost (direct) forecast, first window

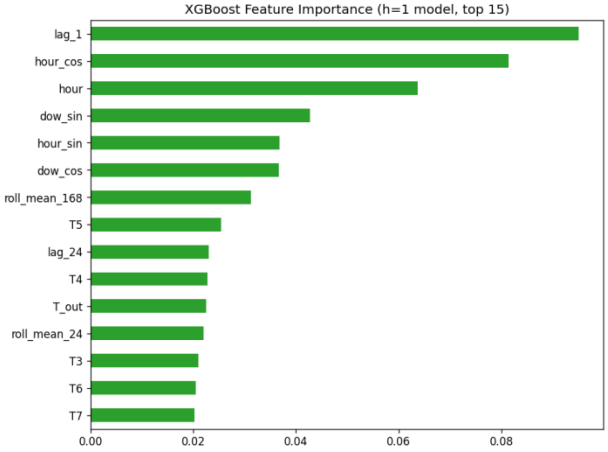

Figure 8: Top 15 XGBoost feature importances

5.4 Foundation model results

Chronos-Bolt-Small, used purely zero-shot, achieved RMSE 334.9 Wh, MAE 201.8 Wh, MAPE 25.8%, and sMAPE 26.0% the best MAE, MAPE and sMAPE of any model tested, and within 2% of SARIMA's RMSE, despite receiving no training or fine-tuning on this dataset whatsoever.

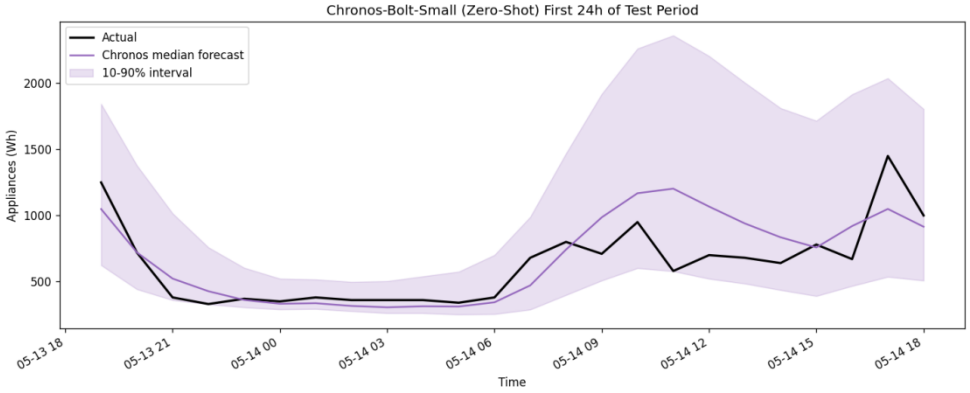

Figure 9: Chronos 24h forecast, first test window

5.5 Consolidated comparison

Table 1: Rolling 14-window average results, all models, sorted by RMSE

Model	RMSE (Wh)	MAE (Wh)	MAPE	sMAPE
SARIMA(1,0,6)×(1,1,1, 24)	328.6	214.2	33.1%	29.7%
Chronos-Bolt-Small (zero-shot)	334.9	201.8	25.8%	26.0%
XGBoost (direct multi-horizon)	341.2	240.8	39.6%	34.0%
Mean (benchmark)	391.2	296.1	53.6%	46.0%
Weekly Seasonal Naïve (benchmark)	404.9	254.6	37.1%	32.5%
Daily Seasonal Naive (benchmark)	455.8	288.6	43.8%	35.5%
Naive (benchmark)	584.7	512.0	113.4%	65.4%
Drift (benchmark)	586.1	513.6	113.8%	65.5%

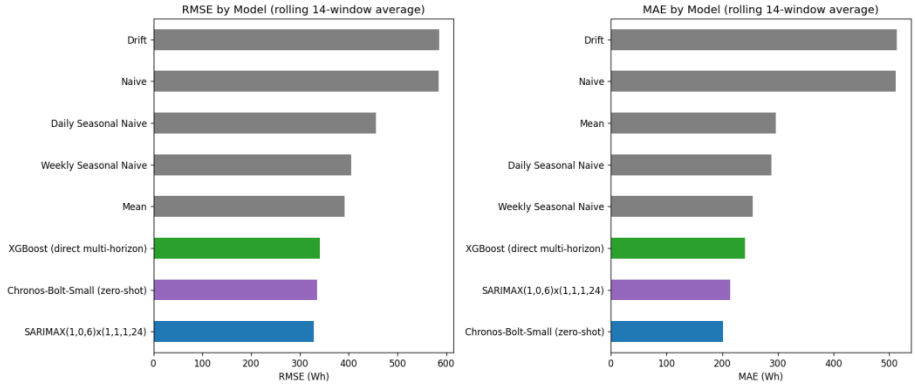

Figure 10: RMSE and MAE comparison across all eight models

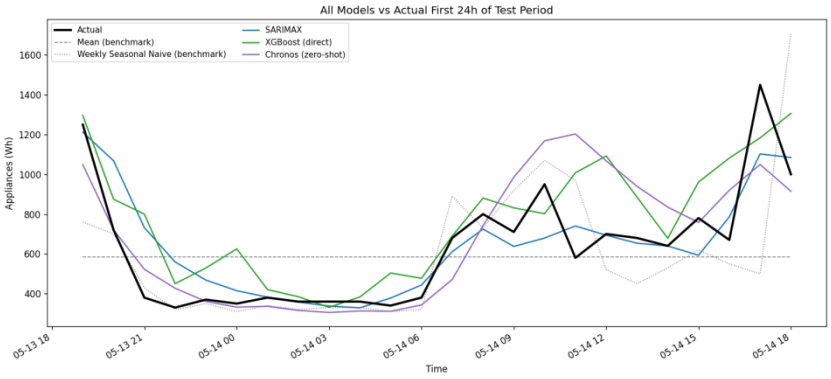

Figure 11: Selected model forecasts vs. actual for the first 24-hour test window

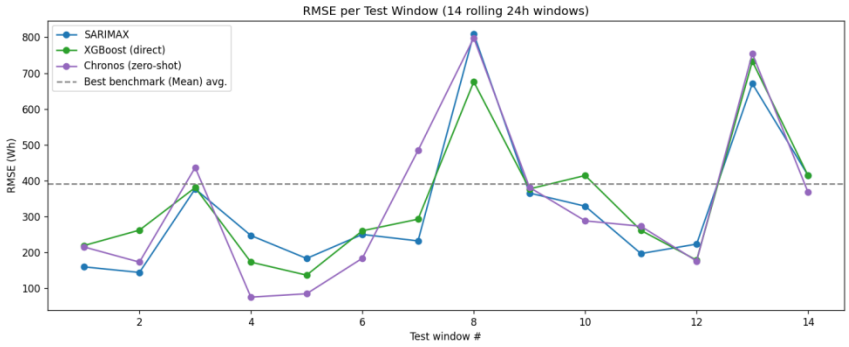

Figure 12: Per-window RMSE across all 14 rolling test windows for the three fitted/pretrained models. All three struggle similarly on windows 8 and 13, indicating genuinely harder days rather than model-specific weakness.

6. Discussion

Q1. Which benchmark model is strongest, and what does this tell you about the structure of appliance energy use?

Among the five named benchmarks (mean, naive, daily seasonal naive, weekly seasonal naive, drift), Weekly Seasonal Naive method perform best overall: achieving the lowest MAE (254.6 Wh), MAPE (37.1%) and sMAPE (32.5%), and second-best RMSE. Naive and Drift perform worst (RMSE ≈ 585), since anchoring on the last value poorly predicts a series with large intra-day swings. Weekly Seasonal Naive beating Daily Seasonal Naive indicates that day-to-day timing of occupant behaviour repeats more reliably at a 7-day lag than a 1-day lag consistent with the mild weekly pattern in Figure 2. Appliance use is therefore seasonal at two nested periods (24h and 168h), and a credible model must capture both.

Q2. Does SARIMA improve on the strongest seasonal benchmark? Are daily seasonality, autocorrelation, and exogenous variables adequately captured?

Yes, decisively: SARIMA's RMSE (328.6 Wh) is ~19% lower than Weekly Seasonal Naive's (404.9 Wh), with matching improvements on every other metric. Daily seasonality and autocorrelation are well captured the residual ACF shows no remaining structure at any lag, and the Ljung-Box test confirms white-noise residuals at lags 24 and

48. Exogenous variables were deliberately excluded (making this a SARIMA rather than a true SARIMAX with exogenous inputs), based on Section 4.3's finding that all sensor/weather correlations are weak ($|r| \leq 0.193$; approximately ≤ 0.19); since SARIMAX assumes a linear exogenous relationship, the expected benefit did not justify the added estimation complexity on a ~3,000-observation series. A version with weather regressors remains a reasonable extension, though the correlation evidence suggests limited expected gain.

Q3. Does the ML model improve with lag, rolling, time, and sensor/weather features? Which feature groups are most useful?

Yes, considerably, but strategy mattered as much as features: the recursive approach suffered from compounding error (RMSE 394.5, worse than most benchmarks), while the direct multi-horizon strategy performed much batter, reaching an RMSE 341.2 Wh, beating every simple benchmark on RMSE and MAE. Feature importance ranks lag_1 highest, followed closely by cyclical hour-of-day encodings, then day-of-week and weekly rolling features; individual weather/sensor variables rank well below these, contributing only marginal non-linear signal. This confirms that when something happens (recency + time-of-day + day-of-week) is far more predictive here than ambient conditions.

Q4. Does the foundation model outperform the benchmarks, SARIMA, and the feature-based model? Is any improvement large enough to justify the added complexity?

Chronos beats every benchmark and XGBoost on every metric, and beats SARIMA on three of four metrics (MAE 201.8 vs 214.2, MAPE 25.8% vs 33.1%, sMAPE 26.0% vs 29.7%), losing only narrowly on RMSE (334.9 vs 328.6, ~2%). This is achieved with zero training or fine-tuning on this dataset. We argue the added architectural complexity is justified here: while Chronos is the most complex model internally (a pretrained transformer vs. SARIMA's), it required the least user-side effort to deploy, no feature-engineering pipeline, just a single inference call while matching or exceeding both fitted alternatives on typical-case accuracy.

Q5. Which variables would genuinely be known at the forecast origin? Is the ML forecast a true or conditional forecast?

At the forecast origin, both short lags (lag_1, lag_2, lag_3) and longer lags (lag_24, lag_48, lag_168) are available from the observed history. However, the short-lag values cannot be directly updated for future forecast steps because they would depend on observations that have not yet occurred. The recursive XGBoost strategy addresses this by using future feature rows, effectively assuming that future sensor/weather information is available, and therefore represents an idealized conditional forecasting scenario. The final direct multi-horizon XGBoost strategy instead trains a separate model for each forecast horizon using only the feature vector available at the forecast origin, with no access to future test-set covariates. Therefore, the reported 341.2 Wh RMSE reflects an origin-time feature-based forecast rather than a forecast conditioned on perfect future information. SARIMA and Chronos remain purely target-series-based models, so the feature information differs across the compared approaches; however, the final direct XGBoost model does not use future test-set observations during forecasting.

Q6. Considering accuracy, interpretability, uncertainty, computational cost, and deployment ease, which model would you recommend?

SARIMA offers the highest interpretability (explicit, diagnosable coefficients) and near-best accuracy, at the cost of a one-off but nontrivial order-search and analytic (imperfectly calibrated) confidence intervals. XGBoost, despite requiring the most operational infrastructure (a live weather/sensor feed of debatable real-world legitimacy per Q5), delivered the weakest accuracy of the three fitted/pretrained approaches. Chronos delivered the best typical-case accuracy (MAE/MAPE/sMAPE) with zero training, no exogenous data dependency, and native distribution-free quantile uncertainty, at the cost of complete interpretability and a heavier runtime dependency (a transformer requiring PyTorch).

For a practical smart-home deployment plausibly across many households with limited per-home historical data and no capacity for individual tuning we recommend Chronos as the primary model, given its combination of top-tier typical-case accuracy, zero per-household training, and no weather-feed dependency. SARIMA is recommended as

the fallback where interpretability, auditability, or minimal compute footprint are required. XGBoost is not recommended as the primary model for this specific use case because its additional feature-engineering and model-maintenance complexity is not repaid by superior accuracy: it ranks third behind SARIMA and Chronos on the consolidated metrics. Its feature-based formulation may nevertheless be useful where origin-time sensor/weather information is reliably available or where feature-level analysis is a specific requirement.

7. Limitations and Future Work

1. SARIMA's confidence intervals are based on a Gaussian assumption that the (non-normal, heavy-tailed) residuals do not fully satisfy, understating spike risk and occasionally implying negative energy use.
2. The XGBoost model uses a richer origin-time feature set than SARIMA and Chronos, including sensor/weather variables, time encodings, and target-derived lag/rolling features. Although the final direct model does not access future test-set covariates, its additional feature inputs make the comparison structurally different from the univariate SARIMA and Chronos forecasts. A stricter comparison could evaluate XGBoost using only target-history and deterministic calendar features.
3. The AIC grid search for SARIMA was conducted on a 30-day subset of the training data for computational tractability, rather than the full training set; this is a standard trade-off, but a full-data search cannot be ruled out as changing the selected order slightly.
4. Only one Chronos model variant (Bolt-Small) and one quantile configuration were tested; larger Chronos variants, or ensembling across quantiles/models, may improve on the zero-shot results reported here.
5. The test period covers only 14 days in one season (May); performance across other seasons, holidays, or households with different occupancy patterns is untested.
6. Future work could explore fine-tuning Chronos on a portion of the training data (rather than pure zero-shot), a SARIMA variant with exogenous weather regressors despite the weak linear correlations found, and hyperparameter tuning for the XGBoost model beyond the default settings used here.

8. References

Candanedo, L. M., Feldheim, V., & Deramaix, D. (2017). Data driven prediction models of energy use of appliances in a low-energy house. Energy and Buildings, 140, 81–97.

Ansari, A. F., et al. (2024). Chronos: Learning the Language of Time Series. arXiv:2403.07815.

Hyndman, R. J., & Athanasopoulos, G. (2021). Forecasting: Principles and Practice (3rd ed.). OTexts.

Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and statistical modeling with Python. Proceedings of the 9th Python in Science Conference.

Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining